

ONEDRAFT

CAD-AI — Aria Powered

CI/CD, Build & Release Engineering

Version 1.0

Status: Implementation Specification

Repository: OneDraft

Primary Function: Automated build, test, migration, deployment, versioning and release control

CI/CD Model: Automated Pipeline with Controlled Promotion

Deployment Model: Environment-Promoted Releases

Artifact Model: Immutable Build Artifacts

Database: PostgreSQL

Runtime: Containerized Services

API: REST / JSON

Workers: Asynchronous Job Execution

AI Layer: Aria Powered

1. PURPOSE

The OneDraft CI/CD, Build & Release Engineering subsystem provides the controlled mechanism by which source code becomes a tested, versioned, deployable and reproducible OneDraft software release.

The system shall automate:

source validation

dependency installation

static analysis

type checking

unit testing

integration testing

contract testing

database migration verification

geometry testing

documentation generation testing

container image construction

artifact generation

security scanning

release packaging

deployment

deployment verification

rollback

The objective is to ensure that no software change reaches a production environment without passing the required engineering gates.

2. CORE PRINCIPLE

OneDraft shall treat software releases as controlled computational artifacts.

The fundamental pipeline is:

```
SOURCE
↓
VALIDATE
↓
BUILD
↓
TEST
↓
SCAN
↓
PACKAGE
↓
VERSION
↓
PUBLISH
↓
DEPLOY
↓
VERIFY
↓
RELEASE
```

A failed stage shall prevent promotion to the next protected environment.

3. RELEASE ENGINEERING BOUNDARY

CI/CD is responsible for transforming source into deployable artifacts.

It is not responsible for making domain decisions.

The boundary is:

```
DEVELOPER
↓
SOURCE CONTROL
↓
CI PIPELINE
↓
BUILD ARTIFACT
↓
TEST / VALIDATION
↓
RELEASE ARTIFACT
↓
DEPLOYMENT SYSTEM
↓
RUNTIME
```

The runtime system remains responsible for executing OneDraft.

The CI/CD system remains responsible for proving that the runtime artifact is suitable for deployment.

4. ENVIRONMENT MODEL

OneDraft shall initially define:

```
LOCAL
↓
DEVELOPMENT
↓
TEST
↓
STAGING
↓
PRODUCTION
```

Each environment is independently configurable.

Production configuration shall never be implicitly inherited from development configuration.

5. LOCAL DEVELOPMENT

Local development shall support:

source checkout
dependency installation
local database
local object storage
local queue
local API
local workers
local frontend
local test execution

The local environment should be reproducible through the repository's development configuration.

Conceptually:

`docker compose up`

or equivalent development orchestration shall establish the required infrastructure.

6. DEVELOPMENT ENVIRONMENT

The development environment is the first shared execution environment.

It shall permit:

continuous integration testing

branch deployment

database migration testing

worker testing

API testing

frontend integration

geometry-engine integration

Aria integration

Development deployments may occur automatically following successful CI execution.

7. TEST ENVIRONMENT

The test environment shall provide deterministic automated testing.

It shall contain:

test PostgreSQL
test object storage
test queue

test workers
test API
test geometry services
test documentation services

Production customer data shall never be used as uncontrolled test fixtures.

8. STAGING ENVIRONMENT

Staging shall represent the production architecture as closely as practical.

The staging environment is the final deployment gate before production.

It shall execute:

database migration tests
integration tests
end-to-end tests
security checks
artifact verification
deployment verification
rollback verification

9. PRODUCTION ENVIRONMENT

Production shall contain only release artifacts that have passed the required release gates.

Production deployment shall require:

successful build
successful tests
successful security checks
approved release
validated artifact
successful deployment
successful health checks

10. SOURCE CONTROL

The OneDraft repository shall be the authoritative source for application code, infrastructure definitions, migrations, tests and release configuration.

The repository shall contain, conceptually:

apps/
packages/

```
services/  
workers/  
migrations/  
tests/  
docs/  
infra/  
scripts/  
.github/
```

Generated artifacts shall not replace source-controlled definitions.

11. BRANCH STRATEGY

The initial branch structure shall support:

```
main  
develop  
feature/*  
release/*  
hotfix/*
```

The precise branching strategy may evolve.

The protected production branch shall require successful automated checks before merge.

12. PULL REQUEST VALIDATION

Every pull request affecting executable software should trigger automated validation.

The minimum sequence is:

```
CHECKOUT  
↓  
INSTALL  
↓  
LINT  
↓  
TYPE CHECK  
↓  
UNIT TEST  
↓  
BUILD  
↓  
CONTRACT TEST
```

Additional tests shall execute when affected components require them.

13. CHANGE DETECTION

The pipeline should identify affected components where practical.

For example:

frontend change
→ frontend tests

API change
→ API tests
→ contract tests

geometry change
→ geometry tests
→ model tests
→ regression tests

database migration
→ migration tests
→ integration tests

Critical integration suites shall still execute for protected branches.

14. DEPENDENCY MANAGEMENT

Dependencies shall be explicitly versioned.

The repository shall maintain lock files for supported package ecosystems.

Dependencies shall be:

pinned

auditable

repeatable

upgradeable

Dependency changes shall be included in CI validation.

15. PYTHON BUILD

Python services shall support deterministic dependency installation.

The build system shall validate:

Python version
dependency lock

package metadata
type compatibility
import integrity
test execution

The resulting Python application artifact shall be suitable for containerization.

16. TYPESCRIPT BUILD

Frontend and TypeScript packages shall validate:

Node runtime
package lock
TypeScript compilation
linting
unit tests
API client generation
production bundle

Generated API clients shall be derived from the authoritative OpenAPI contract.

17. DATABASE MIGRATION PIPELINE

Every PostgreSQL migration shall be tested before deployment.

The migration process shall be:

```
CREATE DATABASE
↓
APPLY ALL MIGRATIONS
↓
VERIFY SCHEMA
↓
RUN INTEGRATION TESTS
↓
DESTROY TEST DATABASE
```

Production migrations shall never be tested first against production.

18. MIGRATION ORDER

Migrations remain sequential and immutable.

For example:

001_initial_schema
002_identity

003_projects

...

032_release_metadata

A migration that has been applied to a shared environment shall not be modified in place.

A correction requires a new migration.

19. MIGRATION SAFETY

Production migrations shall be evaluated for:

locking

execution duration

data transformation

rollback requirements

backward compatibility

application compatibility

Large migrations should use expand-and-contract techniques where required.

20. DATABASE COMPATIBILITY

Application deployments must account for the relationship between:

APPLICATION VERSION

DATABASE VERSION

The deployment process shall prevent an application from requiring database structures that do not yet exist.

The preferred sequence is:

BACKWARD-COMPATIBLE DATABASE CHANGE

↓

APPLICATION DEPLOYMENT

↓

DATABASE FINALIZATION

21. STATIC ANALYSIS

CI shall execute static analysis appropriate to each language.

Minimum categories include:

- lint
- format validation
- type checking
- unused dependency detection
- configuration validation

Static analysis failures shall block protected releases.

22. UNIT TESTING

Unit tests shall verify isolated components.

Critical targets include:

- domain models
- command handlers
- constraint evaluation
- geometry utilities
- validation rules
- API serializers
- authorization policies
- job handlers
- event handlers
- document generation components

23. INTEGRATION TESTING

Integration tests shall verify actual subsystem interactions.

Examples:

- API → database
- API → command service
- command service → revision system
- command service → geometry worker
- validation → compliance database
- drawing engine → model
- document engine → artifact storage
- event bus → workers

24. CONTRACT TESTING

The OpenAPI contract shall be tested against actual API implementations.

Contract testing shall verify:

- request schemas
- response schemas
- status codes
- authentication requirements
- error structures
- pagination
- versioning

The implementation shall not silently diverge from the published API contract.

25. GEOMETRY REGRESSION TESTING

Geometry changes require specialized regression testing.

Known model fixtures shall produce expected geometric characteristics.

Tests may verify:

- dimensions
- topology
- connectivity
- bounding boxes
- element counts
- intersections
- clearances
- model hashes

Small numerical differences shall be distinguished from meaningful geometric regressions.

26. DOCUMENTATION REGRESSION TESTING

Drawing and document generation shall be regression-tested.

Fixtures shall verify:

- sheet count
- sheet numbering
- drawing references
- view generation
- dimensions
- annotations
- schedules

title blocks
revision information
document hashes

Where deterministic rendering is available, generated outputs may be compared against approved reference artifacts.

27. END-TO-END TESTING

The CI/CD system shall support the complete OneDraft workflow:

```
CREATE PROJECT
↓
CREATE REQUIREMENTS
↓
CREATE MODEL
↓
CREATE GEOMETRY
↓
RUN COMPLIANCE
↓
RUN VALIDATION
↓
GENERATE DRAWINGS
↓
CREATE REDLINE
↓
EXECUTE ARIA COMMAND
↓
REVALIDATE
↓
REGENERATE DOCUMENTATION
↓
PACKAGE DOCUMENTS
↓
ACCEPT
```

This becomes a major release qualification test.

28. TEST FIXTURES

The repository shall maintain controlled fixtures for:

residential projects

commercial projects

multi-level buildings

wall assemblies

doors

windows

stairs

structural elements

site constraints

code constraints

redlines

drawing sets

Fixtures shall have explicit version identifiers.

29. TEST DATA ISOLATION

Tests shall use isolated data.

Each test environment should support:

- unique database
- unique project identifiers
- isolated object storage
- isolated queue messages

Parallel test execution must not create uncontrolled cross-test dependencies.

30. BUILD ARTIFACTS

Every successful build shall produce identifiable artifacts.

Examples:

- API container image
- worker container image
- frontend bundle
- migration package
- OpenAPI specification
- Python package
- TypeScript package
- documentation artifacts

31. ARTIFACT IMMUTABILITY

Once an artifact is published under a release identifier, it shall not be silently replaced.

The artifact identity shall include:

```
repository  
commit SHA  
build ID  
version  
artifact hash  
build timestamp
```

32. CONTAINER IMAGE VERSIONING

Container images shall use immutable identifiers.

Conceptually:

```
onedraft-api:<version>  
onedraft-api:<commit-sha>
```

The commit SHA provides an immutable deployment reference.

Production should deploy immutable digests where supported.

33. RELEASE VERSIONING

OneDraft shall use semantic versioning for externally meaningful software releases:

MAJOR.MINOR.PATCH

Example:

```
0.1.0  
0.2.0  
0.2.1  
1.0.0
```

Pre-release identifiers may be used during development.

34. RELEASE CANDIDATES

A release candidate shall be created before a production release when appropriate.

Conceptually:

0.8.0-rc.1

The release candidate must use the same artifacts that will be promoted to production.

35. BUILD MANIFEST

Every release shall generate a build manifest.

Conceptually:

```
{
  "version": "0.1.0",
  "commit": "sha",
  "build_id": "uuid",
  "artifacts": [],
  "dependencies": [],
  "migration_version": "025",
  "created_at": "ISO-8601"
}
```

The manifest provides release provenance.

36. RELEASE MANIFEST

The release manifest shall additionally identify:

application version
API version
database migration version
geometry engine version
compliance engine version
documentation engine version
worker version
frontend version
container digests
OpenAPI version

This allows the complete runtime composition to be reconstructed.

37. SECURITY SCANNING

The CI/CD pipeline shall scan:

source dependencies
container images

Python packages
Node packages
infrastructure configuration
secrets
license metadata

Critical security findings shall block production releases according to the configured security policy.

38. SECRET DETECTION

The pipeline shall detect accidental inclusion of:

API keys
tokens
passwords
private keys
cloud credentials
database credentials

Secrets shall never be committed to source control.

39. LICENSE COMPLIANCE

Third-party dependencies shall be tracked.

The build system should identify:

package
version
license
source

Dependencies incompatible with OneDraft's distribution requirements shall require explicit review.

40. CONTAINER BUILD

Container images shall be produced through reproducible build definitions.

The build shall:

install dependencies
copy required source
compile required assets
run tests
set runtime configuration
define health checks
produce immutable image

Build images and runtime images should be separated where practical.

41. MINIMAL RUNTIME IMAGES

Production containers should contain only required runtime dependencies.

Development tools should not be unnecessarily included in production images.

This reduces:

image size

attack surface

startup time

dependency exposure

42. BUILD CACHE

CI shall use caching where it materially improves build performance.

Cache invalidation must account for dependency and source changes.

A stale cache must never determine production correctness.

43. PIPELINE STAGES

The canonical OneDraft pipeline is:

```
SOURCE
↓
DEPENDENCIES
↓
STATIC ANALYSIS
↓
UNIT TESTS
↓
BUILD
↓
INTEGRATION TESTS
↓
CONTRACT TESTS
↓
SECURITY SCAN
↓
PACKAGE
```

↓
PUBLISH
↓
DEPLOY
↓
SMOKE TEST
↓
RELEASE

44. CI FAILURE POLICY

A required stage failure stops the pipeline.

The pipeline shall not report success when required tests have failed.

Warnings may be permitted only where explicitly classified as non-blocking.

45. DEPLOYMENT GATES

Deployment promotion shall use explicit gates.

For example:

DEVELOPMENT
↓
AUTOMATED TESTS
↓
STAGING
↓
E2E TESTS
↓
RELEASE APPROVAL
↓
PRODUCTION

46. AUTOMATED DEPLOYMENT

Development and test deployments may be fully automated.

Production deployment may initially require an explicit release approval.

As operational confidence increases, selected production deployments may become automatically promoted according to policy.

47. DEPLOYMENT STRATEGY

OneDraft shall support controlled deployment strategies.

Initial strategy:

rolling deployment

Future strategies may include:

blue/green

canary

feature-gated release

48. HEALTH CHECKS

Every deployable service shall expose appropriate health information.

At minimum:

liveness

readiness

version

dependency state

The deployment controller shall distinguish:

process alive

from:

service ready

49. SMOKE TESTS

Immediately following deployment, smoke tests shall verify:

API reachable

authentication functioning

database connectivity

queue connectivity

object storage connectivity

worker availability

basic project retrieval

basic model operation

Failure triggers deployment failure handling.

50. ROLLBACK

A failed deployment shall support rollback to the last known-good release.

Conceptually:

```
NEW RELEASE
↓
HEALTH FAILURE
↓
STOP PROMOTION
↓
RESTORE PREVIOUS ARTIFACT
↓
VERIFY
```

Rollback shall not rely on rebuilding the previous version.

The previous immutable artifact shall remain available.

51. DATABASE ROLLBACK

Database rollback shall be handled independently from application rollback.

Not every database migration is safely reversible.

Where destructive migration is involved, the release process must provide a tested recovery strategy.

52. FEATURE FLAGS

Feature flags may be used to separate:

deployment

from:

feature activation

This allows new functionality to be deployed without immediately exposing it to all users.

Feature flags shall themselves be versioned and auditable.

53. RELEASE APPROVAL

A production release record shall contain:

release version
commit SHA
build ID
artifact hashes
test status
security status
migration status
approval status
deployment timestamp

54. RELEASE RECORD

The system should maintain a persistent release record.

Conceptually:

release_id
version
commit_sha
build_id
artifact_manifest
migration_version
status
approved_by
approved_at
deployed_at

This record becomes part of operational provenance.

55. RELEASE AUDIT

Every production deployment shall create an audit event.

The event shall identify:

release
environment
actor
timestamp
deployment result
artifact version

56. CI/CD EVENT INTEGRATION

The CI/CD system shall integrate with the OneDraft event architecture.

Events may include:

BUILD_STARTED
BUILD_COMPLETED
BUILD_FAILED
TEST_STARTED
TEST_COMPLETED
RELEASE_CREATED
RELEASE_APPROVED
DEPLOYMENT_STARTED
DEPLOYMENT_COMPLETED
DEPLOYMENT_FAILED
ROLLBACK_STARTED
ROLLBACK_COMPLETED

57. JOB INTEGRATION

Build and deployment operations may execute through the existing OneDraft job infrastructure where appropriate.

Long-running operations shall not depend upon an HTTP request remaining open.

58. ARTIFACT STORAGE

Build artifacts shall be stored in controlled artifact storage.

The artifact record shall contain:

artifact_id
artifact_type
version
hash
storage_reference
created_at
retention_policy

This integrates with the OneDraft Artifact & Storage Management Engine.

59. OPENAPI RELEASE

Every API release shall publish the corresponding OpenAPI specification.

The release shall identify:

API version
application version
OpenAPI version
commit SHA

The OpenAPI contract must correspond to the deployed implementation.

60. DATABASE SCHEMA RELEASE

The production environment shall expose a machine-readable migration version.

Conceptually:

```
database_schema_version = 025
```

The application may verify compatibility at startup.

61. SERVICE VERSION REPORTING

Every service shall expose its running version.

For example:

```
{  
  "service": "onedraft-api",  
  "version": "0.1.0",  
  "commit": "sha",  
  "build": "uuid"  
}
```

This supports operational diagnosis.

62. REPRODUCIBLE BUILD REQUIREMENT

A release must be reproducible from:

```
source commit  
dependency lockfiles  
build configuration  
container definition  
migration state
```

The goal is that the same source state produces an equivalent release artifact.

63. BUILD ENVIRONMENT

The CI environment itself shall be controlled.

It shall define:

OS/runtime version
Python version
Node version
package manager version
container tooling
database test version

Uncontrolled developer-machine dependencies shall not determine production builds.

64. PIPELINE CONFIGURATION

Pipeline definitions shall reside in source control.

For example:

`.github/workflows/`

or the equivalent CI platform configuration.

Pipeline changes shall undergo the same review discipline as application code.

65. BUILD SECRETS

CI secrets shall be provided through the CI secret-management mechanism.

The pipeline shall never print secrets to logs.

Logs shall redact sensitive values.

66. ENVIRONMENT PROMOTION

Promotion shall move the same immutable artifact between environments.

Preferred model:

BUILD ONCE
↓
TEST
↓
PROMOTE

↓
PROMOTE
↓
PROMOTE

The artifact should not be rebuilt separately for production.

67. CONFIGURATION PROMOTION

Application artifacts remain identical across environments.

Environment-specific behavior comes from controlled configuration.

Thus:

SAME ARTIFACT
+
DIFFERENT CONFIGURATION
=
DIFFERENT ENVIRONMENT

68. DEPLOYMENT ORDER

A multi-service OneDraft deployment should use an explicit dependency-aware order.

Conceptually:

DATABASE COMPATIBILITY
↓
CORE SERVICES
↓
API
↓
WORKERS
↓
DOCUMENTATION SERVICES
↓
FRONTEND
↓
HEALTH VERIFICATION

The exact sequence may evolve with implementation.

69. WORKER DEPLOYMENT

Workers shall be version-compatible with the API and job schema.

A worker must not process jobs using an incompatible command or payload schema.
Job contracts shall therefore be versioned.

70. EVENT BUS DEPLOYMENT

Event schemas shall be compatible across service versions during rolling deployments.
Breaking event changes require versioned event contracts.

71. FRONTEND DEPLOYMENT

The frontend shall identify the API version it expects.
The frontend build shall verify compatibility against the published API contract.

72. MIGRATION GATE

A deployment that introduces required schema changes shall not proceed until the migration has successfully completed.
The pipeline shall verify:
expected schema version
actual schema version
application compatibility

73. RELEASE GATE MATRIX

The initial release gate matrix is:

Build	REQUIRED
Lint	REQUIRED
Type Check	REQUIRED
Unit Tests	REQUIRED
Integration Tests	REQUIRED
Contract Tests	REQUIRED
Security Scan	REQUIRED
Migration Test	REQUIRED
Geometry Regression	REQUIRED for geometry changes
Documentation Test	REQUIRED for documentation changes
E2E Test	REQUIRED for release
Artifact Verification	REQUIRED

74. CODE COVERAGE

Code coverage shall be measured.

Coverage thresholds shall be established based on risk rather than arbitrary numerical targets.

Critical domain services should receive stronger coverage requirements than low-risk presentation code.

75. REGRESSION SUITE

The regression suite shall become progressively representative of the entire OneDraft platform.

It shall eventually cover:

- project creation
- requirements
- geometry
- constraints
- compliance
- validation
- ARIA commands
- revisions
- redlines
- drawings
- documents
- acceptance
- events
- authorization
- billing
- tenant isolation

76. PERFORMANCE TESTING IN CI

Selected performance tests may execute in CI.

Examples:

- API latency
- database query performance
- geometry computation
- queue throughput
- document generation

Full-scale load testing may execute separately from ordinary pull-request CI.

77. LOAD TESTING

Release candidates should periodically undergo controlled load testing.

Targets include:

- concurrent projects
- concurrent model operations
- geometry workers
- validation workers
- document generation
- API concurrency
- database throughput

78. RELEASE OBSERVABILITY

Every release shall be traceable through the Observability subsystem.

Operational telemetry should allow correlation between:

- release
- request
- job
- event
- model revision
- error

This provides cross-stack diagnostic capability.

79. ERROR BUDGET INTEGRATION

Production reliability measurements may eventually inform deployment policy.

For example:

- high error rate
 - ↓
- deployment promotion suspended

The CI/CD system may consume operational readiness signals before automatically promoting releases.

80. EMERGENCY RELEASES

Emergency releases shall use the same fundamental artifact and provenance controls as ordinary releases.

The process may shorten approval stages where necessary, but must retain:

- source identification
- build identification
- artifact hashing
- deployment record
- post-deployment verification
- audit trail

81. HOTFIX PROCESS

The hotfix flow is:

```
IDENTIFY DEFECT
↓
CREATE HOTFIX
↓
RUN TARGETED TESTS
↓
RUN REQUIRED REGRESSION
↓
BUILD IMMUTABLE ARTIFACT
↓
DEPLOY
↓
VERIFY
↓
MERGE FIX BACK
```

The hotfix must not permanently diverge from the primary development line.

82. RELEASE NOTES

Each release shall produce machine-readable and human-readable release information.

Release notes should identify:

- features
- bug fixes
- breaking changes
- database changes
- API changes
- security changes
- migration requirements
- known limitations

83. CHANGELOG

The repository shall maintain a release history.

The changelog shall correspond to released versions and shall not be treated as a substitute for the source-controlled commit history.

84. API BREAKING CHANGE DETECTION

The pipeline should compare the proposed OpenAPI specification against the previous release.

Breaking changes shall be explicitly detected.

Examples include:

- removed endpoint
- removed response field
- incompatible request change
- changed required field
- changed authentication requirement

85. DATABASE BREAKING CHANGE DETECTION

Database migrations shall be reviewed for:

- column removal
- type incompatibility
- constraint changes
- index impact
- foreign-key impact
- data loss

Potentially destructive changes require explicit release controls.

86. DOMAIN MODEL COMPATIBILITY

Changes to the OneDraft domain model must trigger appropriate regression tests.

Particular attention shall be given to:

Project
Building
Level
Space
Element
Constraint
Revision
Model Version
Command
Validation Result
Drawing
Document Package

87. ARIA RELEASE COMPATIBILITY

Changes to Aria command schemas shall be versioned.

The pipeline shall test:

command parsing
target resolution
parameter validation
constraint preservation
revision creation
validation
event generation

Aria must not produce commands that the deployed domain layer cannot safely interpret.

88. ARTIFACT PROVENANCE

Every production artifact must be traceable to:

repository
commit
build
pipeline
tests
release
deployment

This provenance is part of the OneDraft engineering record.

89. SUPPLY-CHAIN INTEGRITY

The release system should eventually support signed artifacts and verifiable provenance.

The objective is to establish:

SOURCE
↓
BUILD
↓
ARTIFACT
↓
SIGNATURE
↓
DEPLOYMENT

90. BUILD ATTESTATION

Future releases should generate build attestations containing:

source commit
builder
dependencies
build process
artifact hash
timestamp

These attestations provide additional supply-chain verification.

91. RELEASE SECURITY BOUNDARY

CI/CD credentials shall have the minimum permissions required.

Build jobs should not automatically possess unrestricted production access.

Production deployment credentials should be isolated from ordinary CI credentials where practical.

92. PRODUCTION ACCESS

Production deployment access shall be controlled.

The release system should distinguish:

build permission
artifact publication permission
deployment permission
rollback permission
administrative permission

93. ADMINISTRATIVE OVERRIDE

Emergency administrative overrides may exist but must be:

authenticated
authorized
logged
auditable
time-bounded where practical

An override shall not silently bypass the audit system.

94. RELEASE LOCK

A production deployment may establish a release lock preventing simultaneous conflicting deployments.

Conceptually:

DEPLOYMENT LOCK
↓
RELEASE
↓
VERIFY
↓
UNLOCK

95. CONCURRENT DEPLOYMENTS

The system shall prevent incompatible releases from deploying simultaneously.

This is particularly important where:

database migrations
event schemas
job schemas
API contracts

are changing.

96. DEPLOYMENT VERIFICATION

After deployment, the system shall verify:

service health

database compatibility
API availability
worker availability
queue processing
event processing
artifact accessibility

97. AUTOMATIC FAILURE DETECTION

Where practical, deployment monitoring shall detect:

HTTP error increases
worker failures
queue backlog
database failures
startup failures
health-check failures

A failed deployment may automatically halt further promotion.

98. POST-DEPLOYMENT VALIDATION

Production deployment shall not be considered complete merely because containers started.

The system must verify functional availability.

For example:

```
GET health
↓
authenticate
↓
retrieve test resource
↓
submit controlled operation
↓
verify response
```

Production smoke tests shall use safe non-destructive operations.

99. RELEASE COMPLETION

A release becomes:

DEPLOYED

only after required post-deployment checks succeed.

If checks fail:

FAILED

or:

ROLLED_BACK

must be recorded.

100. CI/CD SYSTEM SUCCESS CRITERIA

The OneDraft CI/CD system is successful when it can:

Build the complete application from source

Install deterministic dependencies

Run automated tests

Validate the OpenAPI contract

Validate database migrations

Run geometry regression tests

Run documentation regression tests

Build immutable artifacts

Scan dependencies and containers

Generate release manifests

Publish artifacts

Deploy to development

Promote to staging

Verify staging

Deploy to production

Verify production

Record deployment provenance

Rollback a failed release

101. FIRST AUTOMATED RELEASE TEST

The first complete CI/CD integration test shall execute:

1. Checkout repository
2. Install dependencies
3. Validate source
4. Run linting
5. Run type checks
6. Run unit tests
7. Build application
8. Start PostgreSQL
9. Apply migrations
10. Run database verification
11. Start API
12. Start workers
13. Run integration tests
14. Validate OpenAPI contract
15. Run geometry tests
16. Run documentation tests
17. Run end-to-end project workflow
18. Build production containers
19. Scan artifacts
20. Generate build manifest
21. Publish artifacts
22. Deploy test environment
23. Run smoke tests
24. Generate release candidate
25. Deploy staging
26. Run staging validation
27. Verify release artifacts

28. Record release
29. Execute controlled deployment
30. Run post-deployment health checks
31. Record deployment result
32. Verify rollback capability

This becomes the first major OneDraft release-engineering regression test.

102. REPOSITORY ARTIFACTS

The implementation package shall eventually contain:

```
.github/  
  workflows/  
    ci.yml  
    test.yml  
    build.yml  
    release.yml  
    deploy.yml  
  
infra/  
  docker/  
  environments/  
  deployment/  
  
scripts/  
  build/  
  test/  
  migrate/  
  release/  
  
release/  
  manifests/  
  changelog/  
  
docs/  
  release/
```

The exact directory structure may evolve with the infrastructure implementation.

103. PIPELINE ARTIFACTS

The pipeline shall produce:

```
build manifest  
test report  
coverage report
```

security report
migration report
OpenAPI artifact
container images
release manifest
deployment record

104. MACHINE-READABLE RELEASE RECORD

A release record shall be serializable.

Conceptually:

```
{
  "release_id": "uuid",
  "version": "0.1.0",
  "commit_sha": "sha",
  "build_id": "uuid",
  "migration_version": "025",
  "artifacts": [],
  "tests": {
    "unit": "PASSED",
    "integration": "PASSED",
    "e2e": "PASSED"
  },
  "security": "PASSED",
  "deployment": "PASSED"
}
```

105. RELEASE PROVENANCE CHAIN

The completed release chain shall be:

```
SOURCE COMMIT
↓
BUILD
↓
TEST RESULTS
↓
SECURITY RESULTS
↓
BUILD ARTIFACT
↓
RELEASE MANIFEST
↓
DEPLOYMENT
↓
RUNTIME
```

The system must preserve the relationship between every stage.

106. RELATIONSHIP TO EXISTING ONEDRAFT STACK

The CI/CD subsystem shall consume and integrate with:

Project Architecture
Geometry Engine
Compliance Engine
Validation Engine
Documentation Engine
Review & Acceptance Engine
Runtime Orchestrator
Job & Worker System
Artifact & Storage System
Integration & Event Bus
Identity & Security System
Observability System

It shall not redefine these subsystems.

107. DEPENDENCY DIRECTION

The preferred dependency direction is:

SOURCE
↓
BUILD SYSTEM
↓
APPLICATION ARTIFACT
↓
DEPLOYMENT SYSTEM
↓
RUNTIME

CI/CD must not become a hidden runtime dependency.

OneDraft must remain capable of operating after deployment without the CI/CD platform being continuously available.

108. OPERATIONAL SEPARATION

Build-time operations and runtime operations shall remain distinct.

Build-time:

compile
test
package
scan
release

Runtime:

serve
compute
validate
generate
queue
store
audit

109. RELEASE SEPARATION

A release is immutable after publication.

If a defect is discovered:

NEW SOURCE CHANGE
↓
NEW BUILD
↓
NEW VERSION
↓
NEW ARTIFACT

The previous release remains historically identifiable.

110. VERSIONED INFRASTRUCTURE

Infrastructure definitions shall be source-controlled alongside application architecture.

Changes to:

containers
networking
database
queues
storage
deployment
environment configuration

shall pass CI validation.

111. INFRASTRUCTURE TESTING

Infrastructure changes shall be tested before production deployment.

Tests may include:

- configuration validation
- container startup
- network connectivity
- database connectivity
- health checks
- deployment simulation

112. ENVIRONMENT DRIFT DETECTION

The deployment system should eventually detect divergence between declared infrastructure and deployed infrastructure.

Conceptually:

DECLARED STATE
↓
DEPLOYED STATE

Unexpected differences shall be reported.

113. RELEASE DASHBOARD

The platform should eventually expose a release dashboard showing:

- current version
- previous version
- build status
- test status
- deployment status
- database version
- worker version
- API version
- artifact hashes
- health status

114. DEVELOPER FEEDBACK

CI failures should return actionable information.

A failure should identify:

stage
component
test
error
commit
log reference
recommended investigation target

The system should avoid opaque "build failed" messages.

115. LOG RETENTION

CI/CD logs shall have controlled retention.

Release-critical logs should remain available for the period required by the operational and audit policy.

116. TEST RESULT RETENTION

Test results associated with released artifacts shall remain retrievable.

This permits later reconstruction of:

what was tested
which source was tested
which artifact was produced
which release was deployed

117. RELEASE ARTIFACT RETENTION

Production release artifacts shall be retained according to the Artifact & Storage Management Engine.

At minimum, the system must retain the currently deployed artifact and sufficient previous artifacts to support rollback.

118. ROLLBACK TESTING

Rollback shall itself be tested.

A deployment system that has never successfully executed a rollback shall not be considered operationally complete.

The test sequence is:

```
DEPLOY VERSION A
↓
DEPLOY VERSION B
↓
SIMULATE FAILURE
↓
ROLLBACK TO A
↓
VERIFY
```

119. DISASTER INTEGRATION

CI/CD shall integrate with the Disaster Recovery subsystem.

Release artifacts must remain recoverable even if the primary deployment environment is unavailable.

120. BUILD SYSTEM AVAILABILITY

The loss of CI/CD should prevent new releases but should not prevent the already-deployed OneDraft production system from operating.

This establishes a clean separation between:

SOFTWARE DELIVERY

and:

SOFTWARE EXECUTION

121. RELEASE SECURITY

Production releases shall be traceable to an authorized source-control state.

Untracked binaries shall not be accepted as production releases.

122. RELEASE AUTHORITY

The release process shall distinguish between:

developer
reviewer
release operator
administrator

The precise authorization model shall integrate with the Identity, Access Control & Security System.

123. RELEASE EVENT MODEL

The following events shall be compatible with the OneDraft Event Bus:

BUILD_STARTED
BUILD_PASSED
BUILD_FAILED

TEST_STARTED
TEST_PASSED
TEST_FAILED

ARTIFACT_CREATED
ARTIFACT_PUBLISHED

RELEASE_CREATED
RELEASE_APPROVED
RELEASE_REJECTED

DEPLOYMENT_STARTED
DEPLOYMENT_SUCCEEDED
DEPLOYMENT_FAILED

ROLLBACK_STARTED
ROLLBACK_SUCCEEDED
ROLLBACK_FAILED

124. RELEASE OBSERVABILITY CORRELATION

Every pipeline execution shall have a unique correlation identifier.

That identifier should connect:

commit
build
test
artifact

release
deployment
runtime

This enables end-to-end diagnosis.

125. API / DATABASE / WORKER MATRIX

Each release shall track compatibility between:

API
DATABASE
WORKERS
EVENTS
FRONTEND

Conceptually:

API 0.1
DB 025
Worker 0.1
Events v1
Frontend 0.1

An incompatible combination shall be rejected.

126. RELEASE COMPATIBILITY CHECK

Before deployment, the system shall verify:

API ↔ Database
API ↔ Worker
API ↔ Event Bus
Frontend ↔ API
Worker ↔ Job Schema
Documentation Engine ↔ Model Version

127. BACKWARD COMPATIBILITY

During rolling deployments, services should support compatible versions long enough for the deployment to complete.

Breaking changes require explicit version transitions.

128. RELEASE WINDOW

The platform may eventually support controlled release windows for major infrastructure changes.

Routine automated deployments should not require manual scheduling unless operational policy requires it.

129. CHANGE FREEZE

The production system may support a controlled change freeze.

During a freeze:

ordinary releases

are blocked while:

authorized emergency releases

remain possible.

130. RELEASE POLICY

The initial policy is:

NO TEST

→

NO RELEASE

NO VERIFIED ARTIFACT

→

NO DEPLOYMENT

NO HEALTH CHECK

→

NO COMPLETED DEPLOYMENT

131. MVP IMPLEMENTATION ORDER

The initial implementation should proceed:

1. Repository CI

2. Dependency locking

3. Python tests
 4. TypeScript tests
 5. Database migration tests
 6. Docker builds
 7. API contract tests
 8. Integration tests
 9. End-to-end tests
 10. Artifact publishing
 11. Development deployment
 12. Staging deployment
 13. Production deployment
 14. Rollback
 15. Release provenance
-

132. FIRST RELEASE TARGET

The first production-capable release shall establish:

repeatable build
repeatable test
repeatable migration
repeatable deployment
repeatable rollback

The objective is not merely automated deployment.

The objective is **reproducible software delivery**.

133. ENGINEERING INVARIANTS

The CI/CD system shall preserve the following invariants:

SOURCE IS TRACEABLE

BUILDS ARE REPRODUCIBLE

ARTIFACTS ARE IMMUTABLE

MIGRATIONS ARE VERSIONED

TESTS ARE RECORDED

RELEASES ARE IDENTIFIABLE

DEPLOYMENTS ARE AUDITABLE

ROLLBACKS ARE POSSIBLE

PRODUCTION STATE IS VERIFIABLE

134. ARCHITECTURAL INVARIANT

No production software artifact shall exist outside the release-provenance chain without an explicit and auditable exception.

The preferred path is:

SOURCE

↓

CI

↓

TEST

↓

BUILD

↓

ARTIFACT

↓

RELEASE

↓

DEPLOYMENT

135. RECONSTRUCTION TEST

OneDraft must eventually be capable of reconstructing a production release from:

release_id

version

commit_sha

build_id

artifact_manifest

dependency_lockfiles

migration_version

configuration_version

deployment_record

The reconstruction process must identify exactly what software composition was deployed.

136. MVP CI/CD SUCCESS CRITERIA

The CI/CD implementation is successful when it can:

Detect source changes

Build all required services

Run automated tests

Validate database migrations

Validate OpenAPI contracts

Run geometry regression tests

Run documentation regression tests

Build immutable containers

Scan dependencies

Generate release manifests

Publish artifacts

Deploy to development

Deploy to staging

Deploy to production

Verify deployment health

Record release provenance

Rollback to a previous artifact

137. COMPLETE RELEASE WORKFLOW

The complete OneDraft software delivery process becomes:

DEVELOPER



SOURCE CONTROL



PULL REQUEST



CI VALIDATION



UNIT TESTS



INTEGRATION TESTS



CONTRACT TESTS
↓
GEOMETRY / DOCUMENTATION REGRESSION
↓
SECURITY SCAN
↓
BUILD
↓
ARTIFACT HASH
↓
RELEASE MANIFEST
↓
PUBLISH
↓
DEVELOPMENT
↓
STAGING
↓
END-TO-END VALIDATION
↓
RELEASE APPROVAL
↓
PRODUCTION
↓
SMOKE TEST
↓
HEALTH VERIFICATION
↓
RELEASE COMPLETE

138. RELATIONSHIP TO THE ONEDRAFT PRODUCT

The CI/CD system does not participate directly in architectural design.

Its purpose is to guarantee that the software responsible for:

project modeling
geometry
compliance
validation
documentation
Aria operations
review
acceptance

is delivered consistently and reproducibly.

139. SYSTEM OF DELIVERY RECORD

The release system becomes the system of record for software delivery.

It preserves:

- source provenance
- build provenance
- test provenance
- artifact provenance
- release provenance
- deployment provenance
- rollback provenance

This complements the OneDraft project-model provenance system.

140. FINAL ARCHITECTURAL DECISION

OneDraft shall not depend upon manual software deployment as its primary release mechanism.

It shall be developed as:

- VERSIONED SOURCE
- +
- AUTOMATED BUILD
- +
- AUTOMATED TESTING
- +
- IMMUTABLE ARTIFACTS
- +
- CONTROLLED PROMOTION
- +
- VERIFIED DEPLOYMENT
- +
- AUDITABLE RELEASES
- +
- TESTED ROLLBACK

The CI/CD subsystem therefore becomes the controlled software-delivery mechanism connecting the OneDraft source repository to its executable runtime environments.

141. SPECIFICATION STATUS

ONEDRAFT CI/CD, BUILD & RELEASE ENGINEERING v0.1

STATUS: ESTABLISHED

The next implementation stage is the conversion of this specification into executable delivery infrastructure, beginning with:

CI pipeline definition

Test pipeline definition

Build pipeline definition

Container build definitions

Database migration validation

Artifact publication

Environment deployment definitions

Release manifest generation

Deployment verification

Rollback automation